Domain Primer — What You Need to Know Before the PI Meeting

Everything below is background for the project "AI-Powered Analytics Platform for Requalification of Avionics from Recovered Vehicle Stages." Assumes no prior knowledge. Read Parts 1–4 and 7 if you only have an hour.

Reading plan

Time	Read
30 min	Parts 1, 2, 7 (the problem and the bridge) + Part 10 self-test
60 min	Add Parts 3 and 4 (prognostics + the dataset)
Full	Everything; Parts 5–6 are the engineering vocabulary

Part 1 — The problem: reusable rockets and requalification

1.1 What a launch vehicle stage is

A rocket is built in **stages** — separate propulsion sections stacked on top of each other. Each burns its fuel, then detaches and falls away so the rocket doesn't carry dead weight. The first stage does the heavy lifting off the pad and separates highest-energy and earliest.

Traditionally, the spent stage is destroyed — it falls into the ocean and is written off. That is enormously expensive: you throw away the most complex piece of hardware on every single flight.

1.2 Reuse, and why it creates a new problem

Reusable launch vehicles recover the first stage — by propulsive landing (SpaceX Falcon 9) or by parachute/splashdown recovery — then refurbish and refly it. ISRO's **NGLV** (Next Generation Launch Vehicle) is India's planned heavy-lift vehicle designed with a recoverable first stage. That is the context the problem statement names explicitly.

Recovery creates a question that does not exist for expendable rockets:

> **This hardware flew once and came back. Is it safe to fly again?**

Answering it is called **requalification** (or recertification).

1.3 What requalification actually means

Qualification is the process, before first flight, of proving a component meets its requirements — vibration testing, thermal-vacuum cycling, shock testing, burn-in. It is slow and expensive and done once per design.

Requalification is proving a *specific, already-flown unit* is still fit for another mission. The difference matters:

	Qualification	Requalification
Subject	A design	One individual serial-numbered unit
Question	"Does this design meet spec?"	"Is *this box* still good after what it just went through?"

	Qualification	Requalification
Evidence	Ground test campaign	Flight telemetry + post-recovery inspection
Frequency	Once	Every recovery, every unit

Today this is largely **manual**: engineers inspect hardware and read through flight data by hand. That's slow, doesn't scale to a high flight rate, and is vulnerable to human error. Automating it is the entire point of the problem statement — it says so directly: *"minimize manual inspection time and human errors."*

1.4 The three-way verdict

The problem statement names three focus areas, which are really three severity levels:

Focus area	Meaning	Rough action
Severe damage	The unit is compromised	Reject / scrap
Performance deviations	Measurably off-nominal but maybe acceptable	Needs a human decision
Minor anomalies	Small oddities, probably fine	Clear, but watch it

1.5 The cost asymmetry — the single most important idea in this project

The two ways to be wrong are **not equally bad**:

Error	Consequence
Scrap a good box (too conservative)	You lose the cost of one avionics unit
Clear a bad box (too optimistic)	It fails in flight — you can lose the mission and the payload

Being wrong in the optimistic direction costs orders of magnitude more. Every metric, every model-selection choice, every threshold in this project should reflect that. When you see the "asymmetric score" in Part 3, this is why it exists.

Part 2 — Avionics and how electronics die

2.1 What avionics is

Avionics = *aviation electronics*. On a launch vehicle it's everything electronic that flies: flight computers, navigation sensors (gyros, accelerometers), telemetry transmitters, power distribution, actuator control, sequencing and staging logic, wiring harnesses and connectors.

LRU (Line-Replaceable Unit) — a self-contained box designed to be swapped as a whole. The natural unit of "is this good or not."

2.2 Telemetry

Telemetry is measurement data transmitted from the vehicle to the ground during flight. Two flavours, and the problem statement treats them differently:

Type	Example	What you check
Continuous / analog	Rail voltage, board temperature, current draw, vibration, acoustics, shock	Is the value inside its allowed range? → **bound check**
Fixed / discrete	Health bytes, status flags, mode words, counters	Is this bit pattern legal? → **pattern check**

2.3 Health bytes and BIT

Health bytes are packed status words in the telemetry stream — individual bits, each meaning something: "power supply OK," "self-test passed," "watchdog fired," "memory error detected," plus counters like number of resets.

BIT (Built-In Test) is the hardware's self-diagnosis:

Kind	When it runs
PBIT — power-on	At startup
CBIT — continuous	Constantly in the background
IBIT — initiated	On command

2.4 Bound checking vs pattern checking

These are two different verification ideas and the problem statement names both:

	Bound check	Pattern check
Applies to	Continuous values	Discrete bit patterns / state
Asks	"Is x between min and max?"	"Is this state legal, and was this transition legal?"
Catches	Out-of-range readings	Impossible combinations — e.g. `BIT_PASS` set *while* `SELFTEST_TIMEOUT` is also set
Nature	Threshold comparison	State machine / rule validation

Both are **deterministic** — fixed rules, no learning. The problem statement wants these to work *alongside* ML, each complementing the other. Deterministic checks are explainable and auditable; ML catches things no one wrote a rule for.

2.5 How electronics actually degrade

You need this because it's what makes the analogy to engines defensible or not.

Mechanism	What happens	Model that describes it
Thermal cycling	Repeated heat-up/cool-down; materials expand at different rates; solder joints crack	**Coffin–Manson** (low-cycle fatigue)
Thermal ageing	Chemical degradation accelerated by temperature	**Arrhenius** — rate $\propto \exp(-Ea/kT)$; rough rule of thumb, ageing roughly doubles per +10 °C

Mechanism	What happens	Model that describes it
Vibration fatigue	Random/resonant vibration cracks leads, joints, mounts	Steinberg / PSD-based fatigue models
Radiation — TID	Total ionizing dose accumulates, shifts transistor thresholds	Cumulative dose models
Radiation — SEU	Single-Event Upset: one particle flips one memory bit. *Soft* — recoverable	Probabilistic, rate per bit-day
Radiation — latch-up (SEL)	Particle triggers a parasitic structure into high current; needs a power cycle; can be destructive	Probabilistic
Connector intermittents	Contacts fret, oxidize, momentarily open	Hard to model
Mechanical shock	Stage separation, splashdown — a single violent event	Shock response spectrum

The key split to have in your head:

- **Gradual/cumulative** — thermal cycling, ageing, vibration fatigue, TID. These *accumulate* and in principle you can watch them coming.
- **Abrupt/stochastic** — SEUs, latch-ups, connector intermittents. These strike out of nowhere with no build-up.

Prognostics (Part 3) handles the first kind well and the second kind essentially not at all. This is the main limitation of the whole project, and you should say it before anyone asks.

Note that **Arrhenius and Coffin–Manson are also discussed in the C-MAPSS source paper** — that is a genuine connective tissue between the engine world and the electronics world, and it's a real point in the project's favour.

Part 3 — Prognostics (PHM): the field this project lives in

3.1 The vocabulary

PHM — Prognostics and Health Management. The discipline of estimating the current health of a machine and predicting its future failure, so maintenance is done when needed rather than on a fixed calendar.

Term	Meaning
Diagnostics	What is wrong *now*?
Prognostics	What will happen, and *when*?
RUL — Remaining Useful Life	How much operating life is left before failure. In C-MAPSS, measured in cycles. For avionics: "how many more missions can this box fly?"
Health index	A single number summarizing condition, usually normalized 1 (new) → 0 (failed)
Run-to-failure data	Recordings of units operated until they actually broke. Essential for training, and extremely rare and expensive

Term	Meaning
Condition-based maintenance	Service when the data says to, not on a schedule

3.2 The health-index idea — worth understanding properly

A complex system has several independent ways to fail. Rather than track them separately, define health as the **worst-case margin across all of them**:

> $H(t) = \min(\text{margin}_1, \text{margin}_2, \text{margin}_3, \ldots)$, and the system fails when $H(t) = 0$.

In C-MAPSS the margins are fan efficiency, HPC efficiency, HPT efficiency, and EGT (exhaust gas temperature) limit. Whichever runs out first kills the engine.

The same construction translates directly to avionics: requalification margin = min(thermal margin, timing margin, power margin, signal-integrity margin). **This is the most transferable single idea in the project** — remember it, because the PI will probably probe the analogy and this is the strongest answer.

3.3 Why RUL prediction is hard

- Units start with **different unknown initial wear** — nominally identical boxes are not identical.
- Sensors are noisy.
- The operating regime changes what sensors read, independent of health.
- Early in life there is *no degradation signal at all* — a healthy unit looks healthy whether it has 300 cycles left or 100. (This is exactly why the "RUL cap" trick in Part 6 works so well.)

Part 4 — C-MAPSS: the dataset you actually have

4.1 What it is

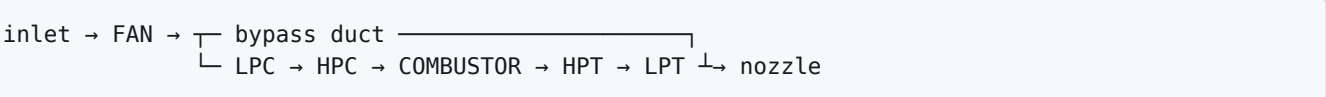
C-MAPSS = Commercial Modular Aero-Propulsion System Simulation — a NASA MATLAB/Simulink model of a large turbofan engine. The dataset is **simulator output**, not measurements from real engines: faults were injected into the model, engines were run until failure, and realistic sensor noise was added.

It is the standard public benchmark for RUL prediction — hundreds of papers use it, so published numbers exist to compare against.

Source paper: **Saxena, Goebel, Simon & Eklund, "Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation," PHM'08.** It's in `data/`. Read it in Week 1.

4.2 Turbofan anatomy — enough to read the sensor names

Air enters the **fan** at the front. It splits: most goes around the core (**bypass** — this is what produces most of the thrust), the rest goes through the core:



Part	Job
Fan	Big front rotor; moves the bypass air
LPC — low-pressure compressor	First compression stage
HPC — high-pressure compressor	Main compression
Combustor	Fuel is burned here
HPT — high-pressure turbine	Extracts energy to drive the HPC
LPT — low-pressure turbine	Extracts energy to drive the fan and LPC

Two independent rotating assemblies (**spools**): **Nf** = fan/low-pressure spool speed, **Nc** = core/high-pressure spool speed.

BPR — bypass ratio: bypass airflow ÷ core airflow. **EGT** — exhaust gas temperature. Its limit is a primary life-limiter on a real engine.

Station numbers. The digits in the sensor names are positions along the gas path (a standard convention): 2 = fan inlet, 24 = LPC outlet, 30 = HPC outlet, 50 = LPT outlet. So `T30` is "temperature at HPC outlet," `P30` is "pressure at the same place." Once you know this the sensor list stops looking like noise.

4.3 The 21 sensors

Name	Meaning
T2	Total temperature at fan inlet
T24	Total temperature at LPC outlet
T30	Total temperature at HPC outlet
T50	Total temperature at LPT outlet
P2	Pressure at fan inlet
P15	Total pressure in the bypass duct
P30	Total pressure at HPC outlet
Nf	Physical fan speed
Nc	Physical core speed
epr	Engine pressure ratio
Ps30	Static pressure at HPC outlet
phi	Fuel flow ÷ Ps30
NRf	Corrected fan speed
NRc	Corrected core speed
BPR	Bypass ratio
farB	Burner fuel-air ratio
htBleed	Bleed enthalpy

Name	Meaning
Nf_dmd	*Demanded* fan speed
PCNfR_dmd	*Demanded* corrected fan speed
W31	HPT coolant bleed flow
W32	LPT coolant bleed flow

"**Corrected**" speed means normalized to standard temperature/pressure so readings are comparable across altitudes. "**Demanded**" means a commanded setpoint, not a measurement — which is why those columns are frozen when the operating condition never changes.

4.4 Data layout

One row = one engine, one flight cycle. 26 space-separated columns:

`unit | cycle | op_setting_1 | op_setting_2 | op_setting_3 | 21 sensors`

The three operating settings are **altitude**, **Mach number**, and **TRA** (throttle resolver angle — the throttle lever position). They are *exogenous*: they change the readings a lot and have nothing to do with health, so they must be normalized out.

4.5 The four datasets

	FD001	FD002	FD003	FD004
Operating conditions	1	6	1	6
Fault modes	1 (HPC)	1 (HPC)	2 (HPC + fan)	2 (HPC + fan)
Train units	100	260	100	**249**
Test units	100	259	100	**248**
Train rows	20,631	53,759	24,720	61,249
Difficulty	easiest	↓	↓	hardest

It's a built-in **difficulty ladder**: FD001 is one condition and one fault; FD004 is six conditions and two faults at once.

(`readme.txt` states FD004 has 248 train / 249 test. The files are the reverse — verified. A nice detail to mention: it shows you opened the data rather than trusting the docs.)

4.6 Train vs test — how the benchmark works

- **Training set:** each engine runs all the way to failure. So RUL at any row = (last cycle of that engine) − (current cycle).
- **Test set:** each trajectory is **cut off at some random point before failure**. You must predict how many cycles remain.
- The true answers for the test set live only in `RUL_FD00x.txt`.

The trap: never derive test labels from how long the test trajectory happens to be — it was truncated arbitrarily. That's **label leakage**, and it silently produces impossibly good numbers.

4.7 How the data was generated

Worth knowing because the whole project's defensibility rests on it:

- Each engine starts with **random unknown initial wear** — normal manufacturing variation, not a fault.
- A fault is injected and grows via an **exponential wear model**: h(t) = 1 − d − exp{a·t^b}, with a ∈ [0.001, 0.003], b ∈ [1.4, 1.6]. So degradation starts almost flat and accelerates.
- Health = min over the operability margins; failure at zero.
- Sensor noise is added on top.

4.8 The PHM'08 scoring function

The competition metric that goes with this dataset. With **d = predicted RUL − true RUL**:

Case	Meaning	Penalty
d < 0	**Early** — you predicted failure sooner than it came. Conservative	exp(−d/13) − 1
d ≥ 0	**Late** — you predicted more life than there was. Dangerous	exp(d/10) − 1

Both are exponential, so large errors are punished disproportionately. The smaller divisor for late errors makes them grow faster — **late predictions cost more.** That's §1.5's asymmetry made into a number, which is exactly why this project uses it instead of RMSE for model selection.

⚠️ **Known wrinkle:** the paper's table gives $a_1 = 10$, $a_2 = 13$ while the text says late errors are penalized harder. The community-standard implementation is 13 early / 10 late. This repo uses the community convention and documents it. Be ready to mention this — it reads as care, not pedantry.

Part 5 — Data engineering vocabulary

5.1 ETL

ETL — Extract, Transform, Load. Pull data from sources, clean and reshape it, write it somewhere queryable. (**ELT** loads first and transforms inside the warehouse — the modern default when the warehouse is powerful.)

5.2 Medallion architecture

The standard layered layout, and what this project proposes:

Layer	Contains	Rule
Bronze	Raw data exactly as received	Immutable. Never edited — if a transform is wrong you can always rebuild
Silver	Typed, validated, conformed	Cleaned, keyed, normalized, joined
Gold	Analytics-ready tables	Features, labels, aggregates — what dashboards and models read

5.3 Storage and query

Term	Meaning
Row-oriented vs **columnar**	Row stores keep a record's fields together (good for "fetch one record"); columnar stores keep each column together (good for "average this column over 10M rows")
Parquet	The standard columnar file format. Compresses well, stores schema, supports selective reads
Predicate pushdown	Filter *while reading* rather than loading everything then filtering. Skips whole chunks
Partitioning	Split files by a key (e.g. by unit, by date) so queries touch only relevant files
OLTP vs **OLAP**	Transactional (many small reads/writes — Postgres) vs analytical (big scans and aggregations — a warehouse)
DuckDB	An in-process analytical database — "SQLite for analytics." No server to run, reads Parquet directly, handles data larger than RAM
Data warehouse / lake / lakehouse	Warehouse = structured and queryable; lake = raw files, cheap and unstructured; lakehouse = lake storage with warehouse behaviour (Iceberg, Delta Lake)
Schema contract	An agreed, documented column structure. Lets you swap the data source without changing downstream code — how real VSSC data would replace the synthetic layer here

5.4 The 64 GB question

The problem statement asks for 64 GB and terabyte-scale storage. C-MAPSS is 27 MB. The reconciliation is that the volume lives in the **high-rate continuous channels**, not in cycle-summary telemetry:

> 3-axis vibration @ 20 kHz × float32 = 240 KB/s → a 10-second burst per mission ≈ 2.4 MB → 100 units × 200 missions ≈ **48 GB** — before acoustics, before imagery, before multiple recovered stages.

Part 6 — Machine learning vocabulary

6.1 The basics

Term	Meaning
Supervised regression	Learn to predict a continuous number (here: RUL) from input features, using labelled examples
Features / target	Inputs / the thing being predicted
Train / test split	Fit on one set, evaluate on data never seen. Otherwise you measure memorization
Overfitting	Model learns noise specific to the training data; looks great in training, fails on new data
Leakage	Information about the answer sneaks into the inputs. Produces suspiciously good results. The #1 thing to guard against here

6.2 The models in play

Model	How it works	Why it's here
Ridge regression	Straight-line fit with a penalty on large coefficients	Simplest possible baseline. Linear — wrong shape for this problem, which is the point
Random forest	Many decision trees on random subsets, averaged	Strong, low-effort nonlinear baseline. Needs no feature scaling and ignores useless columns automatically
1D-CNN / LSTM	Neural networks over *sequences* — a sliding window of recent cycles rather than one row	Where published state-of-the-art lives. Sees degradation *trend*, not just a snapshot
Autoencoder (anomaly)	Learns to reconstruct normal data; anomalies reconstruct badly	Trained only on healthy early cycles — needs no failure labels
Mahalanobis distance (anomaly)	Distance from the healthy distribution, accounting for how variables co-vary	Simpler alternative to the autoencoder

Per-row vs sequence is the distinction worth holding onto: a random forest sees one cycle at a time in isolation; an LSTM sees the recent history. Several results in this project only make sense in that light.

6.3 Evaluation metrics

Metric	Meaning
RMSE	Root Mean Squared Error — typical error size, in cycles. Symmetric: early and late errors cost the same
PHM'08 score	The asymmetric cost above. Lower is better. This is the operationally honest one
Correlation	Do predictions track the truth across units? Included because an aggregate score alone can't distinguish consistently-decent performance from wildly mixed performance

6.4 Two tricks specific to this dataset

The RUL cap (~125 cycles). Raw RUL says an engine at cycle 1 of a 300-cycle life has 299 remaining. But it looks *identical* to a healthy engine with 150 remaining — there is no signal yet. Forcing the model to distinguish them teaches it noise. Fix: clip the target at a ceiling (~125), so every healthy engine gets the same label and the model only learns the part of life where degradation is actually visible. This is standard practice, and in this repo it cut FD001 RMSE from 32.9 to 18.3 and the asymmetric score by 34×.

Operating-condition normalization. In FD002/FD004, six flight regimes make sensors jump between discrete levels that swamp the degradation trend. The standard fix is to cluster rows into their 6 conditions (k-means on the three settings) and normalize each sensor within its own condition. Visually decisive.

But — measured in this repo — it made the random forest slightly *worse*. The reason: the operating settings are themselves columns in the feature matrix, so a tree splits on them and reconstructs the regimes internally; pre-normalizing only throws information away. Published gains come mostly from *sequence* models over sliding windows. Understand this one properly — it's the project's most interesting result so far and it's a negative result with a mechanism, which is a good thing to be able to defend.

Part 7 — The bridge: why turbofan data for an avionics problem?

This is the heart of the project. If you learn one part, learn this.

7.1 The situation

The problem statement is about recovered launch-vehicle avionics. The only dataset available is simulated jet-engine gas-path data. That's not an oversight — **nobody has run-to-failure data for recovered avionics boards.** Reusable stages are new, the flight count is tiny, and no one deliberately flies boxes until they die.

So the project's contribution is not a good RMSE on FD001. It is: **argue rigorously that C-MAPSS is a defensible surrogate, define the mapping precisely, state where it breaks, and design an architecture that survives the swap when real data arrives.**

7.2 What transfers

C-MAPSS	Avionics analogue	Strength
Engine unit	A recovered LRU, by serial number	**Strong** — a fleet of same-type units with unknown initial wear
Cycles	Missions / flight-hours	**Strong**
3 operating settings (altitude, Mach, TRA)	Flight regime: ascent, coast, re-entry, landing	**Strong** — both are exogenous context to normalize out
21 gas-path sensors	Rail voltages, board temperatures, current draw, clock drift, BER, link margin	**Moderate** — both are noisy continuous channels drifting with wear
H(t) = min of operability margins, fail at 0	Requal margin = min(thermal, timing, power, signal-integrity)	**Strong — the most transferable idea in the project**
Exponential wear model	Cumulative damage from thermal cycling, vibration fatigue, radiation dose	**Moderate** — but Arrhenius/Coffin–Manson are discussed in the same paper, which helps
FD001→FD004 difficulty ladder	Single- vs multi-condition, single- vs multi-fault requalification	**Strong**
RUL prediction	"How many more missions can this box fly?"	**Strong**
PHM'08 asymmetric score	Clearing a box that later fails ≫ scrapping a good one	**Strong — the economic core of the problem**

7.3 What does not transfer — say this before you're asked

- Gas-path degradation is **smooth, monotone, thermodynamically constrained**. Avionics also fails **abruptly**: latch-ups, SEUs, solder cracking, connector intermittents. C-MAPSS cannot produce those.
- C-MAPSS assumes damage within a *single* flight is negligible. For a **re-entry and splashdown**, that assumption is questionable — that's one high-stress episode, not gentle accumulation.
- C-MAPSS has no discrete/health-byte channel and no vibration data at all — two of the three things the problem statement explicitly asks for.

Frame these as **limitations that motivate the synthetic layer**, not as fatal flaws.

7.4 Why "it's simulated data" is a strength, not a weakness

C-MAPSS is simulator output, not real engine measurements. That sounds damaging until you read the problem statement, which says:

> *"synthesised data from simulation test beds after fault injection can be provided as inputs."*

So simulated input is **VSSC's own stated data strategy.** Three consequences:

1. The base data is aligned with what the customer expects, not a compromise.
2. Adding a synthetic avionics layer *extends an existing simulation* rather than contaminating real measurements — a far easier position to defend.
3. The honest framing throughout is **"physics-informed simulation surrogate,"** never "real flight data."

7.5 The synthetic avionics layer

C-MAPSS lacks two of the three data classes the problem statement needs, so the project synthesizes them **as functions of the true degradation state** (so they are physically coherent, not independent noise):

Channel	What's generated	Requirement served
Health bytes	A 16-byte status word per mission — BIT pass/fail bits, mode flags, reset counters. Fault-bit flip probability rises as health falls	Pattern checking
Vibration / shock	A short high-rate 3-axis burst per mission. Degradation broadens and shifts resonance peaks and raises the RMS floor; shocks injected at re-entry/landing	Bound checking
Imagery	Proposed only — inspection images of boards. **Explicitly scoped to future work**	Image processing

The rules that keep this defensible (state them or a reviewer will attack the whole thing):

- Every generator is parameterized, with parameters documented and justified against cited literature.
- Synthetic channels demonstrate that the **pipeline mechanics** work — never that the ML is *accurate* on real degradation.
- All degradation-modelling claims rest on the C-MAPSS sensors alone.
- A documented schema contract lets real VSSC channels swap in without touching downstream code.

7.6 The decision framework — the project's novel contribution

Three **independent** evidence streams, fused:

Stream	Produces	Type
Bound + pattern checks	Hard violations, count and severity	Deterministic
RUL model	Predicted remaining missions + uncertainty band	ML
Anomaly detector	Novelty score vs healthy baseline	ML

→ fused into four verdicts: **REJECT** / **CONDITIONAL-CLEAR** (needs human sign-off) / **CLEAR-WITH-MONITORING** / **CLEAR**.

Plus **conditional clearance by precedent.** The problem statement asks for *"conditional clearance based on decisions made during past missions… with human confirmation."* The design makes that concrete as **case-based retrieval**: a decision registry keyed by feature-space neighbourhood, so when a new unit resembles a past case, the reviewer is shown that precedent and what happened. The system **recommends; it never decides.**

Part 8 — Numbers to have memorized

Fact	Value
Total C-MAPSS size	~160k rows, ~27 MB
FD001 train	20,631 rows, 100 units
FD004 train	61,249 rows, 249 units (largest, hardest)
Sensors / settings	21 sensors, 3 operating settings, 26 columns
Operating conditions	1 (FD001, FD003) or 6 (FD002, FD004)
Constant sensors in FD001	6 of 21
RUL cap	~125 cycles
FD001 baseline RMSE (random forest)	18.3, vs 32.9 uncapped
PHM'08 score improvement from capping	34,632 → 1,016
PHM'08 divisors	13 early / 10 late (community convention)
Wear model	h(t) = 1 − d − exp{a·t^b}, a ∈ [0.001, 0.003], b ∈ [1.4, 1.6]
Problem statement data volume	64 GB; terabyte-scale warehouse
Vibration volume argument	20 kHz × 3-axis × float32 → ~48 GB for 100 units × 200 missions

Part 9 — Acronym glossary

BER	Bit Error Rate
BIT	Built-In Test (PBIT power-on, CBIT continuous, IBIT initiated)
BPR	Bypass Ratio
C-MAPSS	Commercial Modular Aero-Propulsion System Simulation
CBM	Condition-Based Maintenance
EGT	Exhaust Gas Temperature
EPR	Engine Pressure Ratio
ETL / ELT	Extract-Transform-Load / Extract-Load-Transform
HPC / HPT	High-Pressure Compressor / Turbine
ISRO	Indian Space Research Organisation

LPC / LPT	Low-Pressure Compressor / Turbine
LRU	Line-Replaceable Unit
NGLV	Next Generation Launch Vehicle (ISRO)
OLAP / OLTP	Online Analytical / Transactional Processing
PHM	Prognostics and Health Management
PSD	Power Spectral Density (vibration)
RMSE	Root Mean Squared Error
RUL	Remaining Useful Life
SEL	Single-Event Latch-up
SEU	Single-Event Upset
TID	Total Ionizing Dose
TRA	Throttle Resolver Angle
VSSC	Vikram Sarabhai Space Centre (ISRO's launch vehicle centre)

Part 10 — Self-test

If you can answer these out loud without notes, you're ready.

1. What is requalification, and how does it differ from qualification?
2. Why is clearing a bad box worse than scrapping a good one — and where does that show up in the metrics?
3. What is the difference between a bound check and a pattern check? Give an example of each.
4. What is RUL, and why is the test set harder than the training set?
5. What does H(t) = min(margins) mean, and why is it the strongest part of the analogy?
6. Name two avionics failure mechanisms that C-MAPSS **cannot** represent.
7. Why is C-MAPSS being *simulated* an argument in the project's favour?
8. Why cap the RUL target at 125? What does it fix?
9. Why did operating-condition normalization make the random forest slightly worse?
10. How do you get from a 27 MB dataset to a 64 GB requirement?
11. What are the three evidence streams in the decision framework, and what four verdicts do they produce?
12. What is bronze/silver/gold, and why must bronze be immutable?